

3.1 Prompt 工程简介

传统的自然语言处理研究遵循“预训练-微调-预测”范式，即先在大规模语料库上作预训练，然后在下游任务上微调，最后在微调后的模型上进行预测。然而，随着语言模型在规模和能力上的显著提升，一种新的范式——“预训练-提示预测”应运而生，即在预训练模型的基础上，通过精心设计 Prompt 引导大模型直接适应下游任务，而无需进行繁琐微调，如图 3.1 所示。在这一过程中，Prompt 的设计将对模型性能产生深远影响。这种专注于如何编写 Prompt 的技术，被称为 **Prompt 工程**。在本节中，我们将深入介绍 Prompt 工程的定义及其相关概念，探讨其在自然语言处理领域中的重要性和应用。

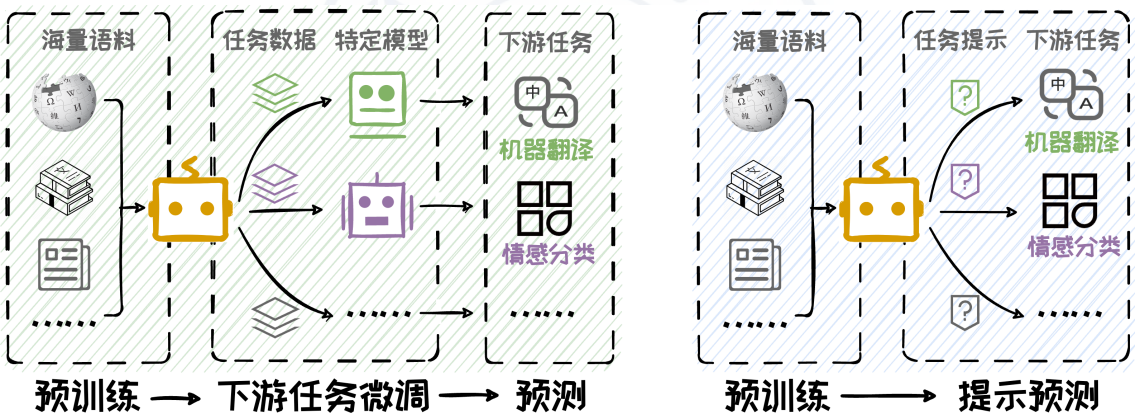


图 3.1: “预训练-微调-预测”范式与“预训练-提示预测”范式对比。

3.1.1 Prompt 的定义

Prompt 是指用于指导生成式人工智能模型执行特定任务的输入指令¹，这些指令通常以自然语言文本的形式出现。Prompt 的核心目的是清晰地描述模型应该执行的任务，以引导模型生成特定的文本、图像、音频等内容。如图 3.2 所示，通过

¹https://en.wikipedia.org/wiki/Prompt_engineering



图 3.2: 几种常见的 Prompt 例子。

精心设计的 Prompt，模型能够实现多样化的功能。例如，通过提供明确的情感分类指令，模型能够准确地对文本进行情感分析；通过特定主题的创作指令，模型能够生成富有创意的诗歌。此外，在多模态模型的应用场景中，Prompt 还可以包含画面描述，从而指导模型生成相应的视觉作品。

Prompt 的应用范围广泛，不仅限于文本到文本的任务。由于本书主要关注语言模型，本章节将聚焦于**文本生成模型**，并深入探讨如何通过精心设计的 Prompt 来引导模型生成符合特定任务要求的文本输出。

3.1.2 Prompt 工程的定义

Prompt 工程 (Prompt Engineering)，又称提示工程，是指设计和优化用于与生成式人工智能模型交互的 Prompt 的过程²。这种技术的核心在于，将新任务通过 Prompt 构建为模型在预训练阶段已经熟悉的形式，利用模型固有的泛化能力来执行新的任务，而无需在额外的特定任务上进行训练。Prompt 工程的成功依赖于对

²https://en.wikipedia.org/wiki/Prompt_engineering

预训练模型的深入理解，以及对任务需求的精确把握。通过构造合适的 Prompt 输入给大语言模型，大语言模型能够帮助我们完成各种任务 [47]。

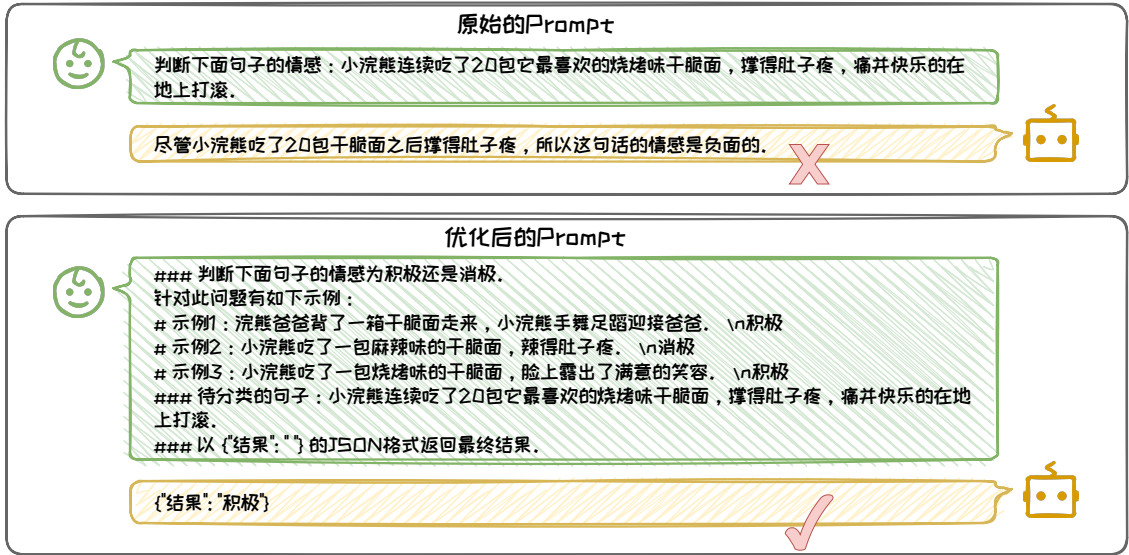


图 3.3: Prompt 工程技术应用前后的效果对比。

如图 3.3所示，通过 Prompt 工程的优化，原始 Prompt 被改写为更加全面、规范的形式。优化后的 Prompt 能够显著提升模型生成回答的质量。因此，在与大语言模型互动过程中构建优质且全面的 Prompt 至关重要，它直接决定了能否获得有价值的输出。经过良好设计的 Prompt 通常由任务说明、上下文、问题、输出格式四个基本元素组成：

- **任务说明**——向模型明确提出具体的任务要求。任务说明应当清晰、直接，并尽可能详细地描述期望模型完成的任务。
- **上下文**——向模型提供的任务相关背景信息，用以增强其对任务的理解以及提供解决任务的思路。上下文可以包括特定的知识前提、目标受众的背景、相关任务的示例，或任何有助于模型更好地理解任务的信息。

- **问题**——向模型描述用户的具体问题或需要处理的信息。这部分应直接涉及用户的查询或任务，为模型提供一个明确的起点。问题可以是显式的提问，也可以是隐式的陈述句，用以表达用户的潜在疑问。
- **输出格式**——期望模型给出的回答的展示形式。这包括输出的格式，以及任何特定的细节要求，如简洁性或详细程度。例如，可以指示模型以 JSON 格式输出结果。

Prompt 的四个基本元素——任务说明、上下文、问题和输出格式，对于大语言模型生成的效果具有显著影响。这些元素的精心设计和组合构成了 Prompt 工程的核心。在此基础上，Prompt 工程包括多种技巧和技术，如上下文学习（In-Context Learning）和思维链（Chain of Thought）等。这些技巧和技术的使用，可以显著提升 Prompt 的质量，进而有效地引导模型生成更符合特定任务需求的输出。具体关于上下文学习的内容将在 3.2 节中讨论，思维链的内容将在 3.3 节中讨论，而 Prompt 的使用技巧将在 3.4 节中详细探讨。

然而，随着 Prompt 内容的丰富和复杂化，输入到模型中的 Prompt 长度也随之增加，这不可避免地导致了模型推理速度的减慢和推理成本的上升。因此，在追求模型性能的同时，如何有效控制和优化 Prompt 的长度，成为了一个亟待解决的问题，需要在确保模型性能不受影响的前提下，尽可能压缩输入到大型模型中的 Prompt 长度。为此，LLMLingua [11] 提出了一种创新的由粗到细的 Prompt 压缩方法，该方法能够在不牺牲语义完整性的情况下，将 Prompt 内容压缩至原来的二十分之一，同时几乎不损失模型的性能。此外，随着 RAG 技术的兴起，模型需要处理的上下文信息量大幅增加。FIT-RAG [22] 技术通过高效压缩检索出的内容，成功将上下文长度缩短至原来的 50% 左右，同时保持了性能的稳定，为处理大规模上下文信息提供了有效的解决方案。

3.1.3 Prompt 分词向量化

在构建合适的 Prompt 之后，用户将其输入到大语言模型中，以期得到满意的生成结果。但是，语言模型无法直接理解文本。在 Prompt 进入大模型之前，需要将它拆分成一个 Token 的序列，其中 Token 是**承载语义的最小单元**，标识具体某个词，并且每个 Token 由 Token ID 唯一标识。将文本转化为 Token 的过程称之为**分词** (Tokenization)，如图 3.4 所示，对于“小浣熊吃干脆面”这样一句话，经过分词处理之后，会变成一个 Token 序列，每个 Token 有对应的 Token ID。

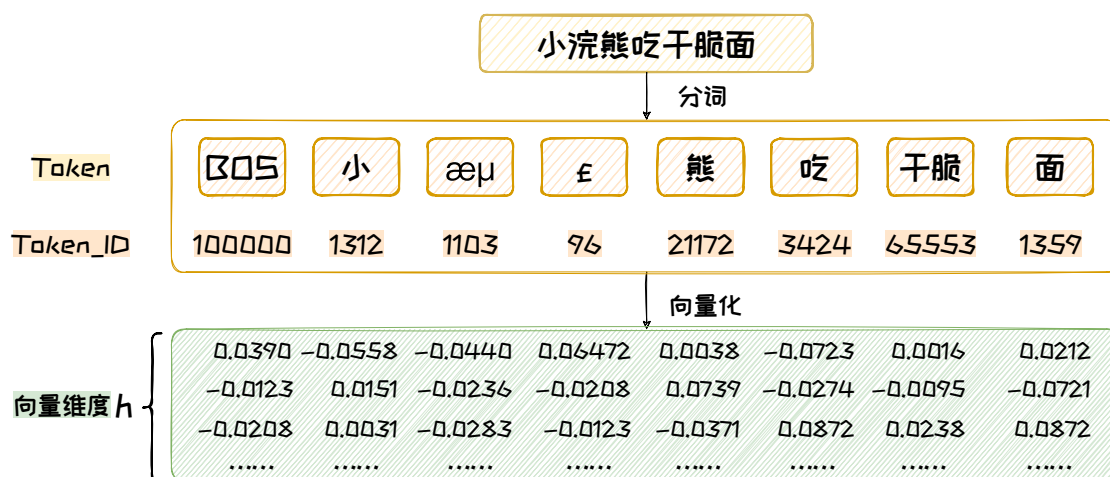


图 3.4: 分词与嵌入的过程，以 DeepSeek-V2-Chat 模型的分词器为例 [6]。

值得注意的是，一句话可能存在多种拆分方式。例如，上述句子可拆分为“小，浣熊，吃干，脆面”。然而，这种拆分方式可能导致语义混乱、不清晰。因此，分词过程颇具挑战性，我们需要精心设计分词方法。为实现有效分词，首先需构建一个包含大语言模型所能识别的所有 **Token** 的**词表**，并依据该词表进行句子拆分。

在构建大语言模型的词表时，分词器依赖于分词算法，如 BBPE [34]、BPE [8] 和 WordPiece [29] 等，这些算法通过分析语料库中的词频等信息来划分 Token。本文将以前面的 BBPE（Byte-Level Byte Pair Encoding）算法为例，阐述其分词过程。BBPE 算法的流程主要包括以下四个步骤：

- **1. 初始化词表**：首先，将所有字符按照其底层编码拆分为若干字节，并将这些单字节编码作为初始词表的 Token。
- **2. 统计词频**：接下来，统计词表中所有 Token 对（即相邻 Token 的组合）的出现频率。在初始阶段，Token 对即为相邻字节的组合。
- **3. 合并高频 Token 对**：然后，选择出现频率最高的 Token 对，将其合并成一个新的 Token 并加入词表。
- **4. 迭代合并**：重复步骤 2 和步骤 3，不断迭代合并，直至达到预设的词表大小或达到指定的合并次数。

我们可以通过设定迭代次数等方法来决定分词粒度，不同的分词粒度导致我们得到不同大小的词表。若**词表过小**，例如仅包含 256 个 Token 表示单字节，能够组合出所有 GBK 编码的汉字和英文，但会导致形态相近但意义迥异的词汇在模型中难以区分，限制了模型承载丰富语义的能力，并大幅增加生成的序列长度。相反，若**词表过大**，虽然能涵盖更多长尾词汇，但可能导致对这些词汇的学习不够深入，且难以有效捕捉同一单词不同形态之间的关联。因此，在构建词表时，需在涵盖广泛词汇与保持语义精细度之间找到恰当的平衡点，以确保大语言模型既能学习到丰富的词汇知识，又能准确理解和生成具有复杂语义的文本。

具体来说，如图 3.4 所示，为了有助于模型更准确地理解词义，同时减少生成常用词所需的 Token 数量，词表中收录了语料库中**高频**出现的词语或短语，形成独立的 Token。例如，“干脆”一词在词表中以一个 Token 来表示。为了优化 Token 空间并压缩词表大小，构建词表时会包含了一些特殊的 Token，这些 Token 既能单独表示语义，也能通过两两组合，表示语料库中**低频**出现的生僻字。例如，“浣”字使用两个 Token，“æµ”和“£”，来表示。通过这种处理方式，词表既能涵盖常见的高频词汇，又能通过 Token 组合灵活表达各类稀有字符。由此可见，词表构建在一定程度上受到“先验知识”的影响，这些知识源自人类语料库的积累与沉淀。

每个大语言模型都有自己的分词器，分词器维护一个词表，能够对文本进行分词。分词器的质量对模型的性能有着直接的影响。一个优秀的分词器不仅能显著提升模型对文本的理解能力，还能够提高模型的处理速度，减少计算资源的消耗。一个好的分词器应当具备以下特点：首先，它能够准确地识别出文本中的关键词和短语，从而帮助模型更好地捕捉语义信息；其次，分词器的效率直接影响到模型的训练和推理速度，一个高效的分词器能够实现对文本 Token 的优化压缩，进而显著缩短模型在处理数据时所需的时间。

表 3.1: 模型分词器对比表

模型	词表大小	中文分词效率 (字 / Token)	英文分词效率 (词 / Token)
LLaMA1	32000	0.6588	0.6891
LLaMA2	32000	0.6588	0.6891
LLaMA3	128256	1.0996	0.7870
DeepSeek-V1	100016	1.2915	0.7625
DeepSeek-V2	100002	1.2915	0.7625
GPT-3.5 & GPT-4	100256	0.7723	0.7867
GPT-3	50,257	0.4858	0.7522
Qwen-1.5	151646	1.2989	0.7865
StarCoder	49152	0.9344	0.6513

在常用的开源模型中，不同模型采用了不同的分词器，这些分词器具有各自的特点和性能。它们的质量受到多种因素的影响，包括词表的大小、分词的效率等属性。表 3.1 是对常见开源大语言模型的分词器的对比分析，其中中文语料库节选自《朱自清散文》³，英文语料库来自莫泊桑短篇小说《项链》⁴。我们可以观察到，像 DeepSeek [6]、Qwen [41] 这类中文开源大语言模型，对中文分词进行了优化，平均每个 Token 能够表示 1.3 个字（每个字仅需 0.7 个 Token 即可表示），一些常用词语和成语甚至可以直接用一个 Token 来表示。相比之下，以英文为主要语料的模型，如 GPT-4、LLaMA 系列，对中文的支持度较弱，分词效率不高。在英文

³https://www.sohu.com/a/746456997_120075260.

⁴<https://americanliterature.com/author/guy-de-maupassant/short-story/the-diamond-necklace>

中，由于存在“ly”、“ist”等后缀 Token，一个英文单词通常需要用 1 个及以上的 Token 来表示。单个 Token 承载更多的语义，模型在表达同样的文本时，只需要输出更少的 Token，显著提升了推理效率。

通过这种对比，我们可以清晰地看到不同模型分词器在处理不同语言时的效率，这对于选择合适的模型和优化模型性能具有重要的指导意义。

在完成分词之后，这些 Token 随后会经过模型的嵌入矩阵 (Embedding Matrix) 处理，转化为固定大小的**表征向量**。这些向量序列被直接输入到模型中，供模型理解和处理。在模型生成阶段，模型会根据输入的向量序列计算出词表中每个词的概率分布。模型从这些概率分布中选择并输出对应的 Token，这些 Token 再被转换为相应的文本内容。

上述通过分词技术将文本分割成 Token，再将 Token 转化为特征向量，在高维空间中表征这些文本的处理流程，使得语言模型能够捕捉文本的深层语义结构，并有效地处理和学习各种语言结构，从简单的词汇到复杂的句式和语境。

3.1.4 Prompt 工程的意义

Prompt 工程提供了一种高效且灵活的途径来执行自然语言处理任务。它允许我们无需对模型进行微调，便能有效地完成既定任务，避免微调带来的巨大开销。通过精心设计的 Prompt，我们能够激发大型语言模型的内在潜力，使其在**垂域任务**、**数据增强**、**智能代理**等多个领域发挥出卓越的性能。

1. 垂域任务

应用 Prompt 工程来引导大语言模型完成垂直领域任务，可以避免针对每个任务进行特定微调。不仅可以避免微调模型的高昂计算成本，还可以减少对标注数据的依赖。使得大语言模型可以更好的应用在垂直领域任务中。例如，在 Text-to-SQL 任务中，我们可以应用 Prompt 工程的技巧，引导大语言模型根据用户输入的文本直接生成高质量的 SQL 查询，而无需进行有监督微调。基于 GPT 模型的 Prompt

工程方法在 Spider [44] 榜单上取得了突破性成绩，超越了传统的微调方法。此外，在知识密集型任务问答领域 MMLU [9] 等多个领域，基于 Prompt 工程的方法也取得最佳效果。

2. 数据增强

应用 Prompt 工程通过大语言模型来进行数据增强，不仅能够提升现有数据集的质量，还能够生成新的高质量数据。这些数据可以用于训练和优化其它模型，以将大语言模型的能力以合成数据的方法“蒸馏”到其他模型上。例如，我们可以引导 ChatGPT 模型生成包含丰富推理步骤的数据集，用于增强金融领域 Text-to-SQL 模型的推理能力 [45]。此外，通过精心设计的提示，还能生成包含复杂指令的数据集，如 Alpaca [32] 和 Evol-Instruct [21]。将这些合成的数据集用于微调参数量较小的模型，可以其在保持较小模型尺寸和低计算成本的同时，接近大型模型的性能。

3. 智能代理

应用 Prompt 工程可以将大语言模型构建为智能代理 (Intelligent Agent, IA)⁵。智能代理，又叫做智能体，能够感知环境、自主采取行动以实现目标，并通过学习或获取知识来提高其性能。在智能代理进行感知环境、采取行动、学习知识的过程中，都离不开 Prompt 工程。例如，斯坦福大学利用 GPT-4 模拟了一个虚拟西部小镇 [25]，多个基于 GPT-4 的智能体在其中生活和互动，他们根据自己的角色和目标自主行动，进行交流，解决问题，并推动小镇的发展。整个虚拟西部小镇的运转都是由 Prompt 工程驱动的。

本节探讨了 Prompt 的概念，Prompt 工程的概念以及意义，揭示了 Prompt 在大语言模型应用中的关键作用和广阔潜力。接下来，我们将进一步拓展这一主题：第3.2节将探索上下文学习的相关内容，揭示其在提升模型理解和响应能力中的作用；第3.3节将详细介绍思维链提示方法及其变种，展示如何通过这些方法增强模

⁵https://en.wikipedia.org/wiki/Intelligent_agent